分子表示比较研究简明汇报

本次结果：输入形式会影响模型成绩；不同任务的最好表示不同，Our Code 并非整体最优。

我们用现有模型 deepseek-v4.1-flash 做统一测试（benchmark）：同一个分子换种写法，问相同问题，用 RDKit 化学工具算出的参考答案评分。模型不调用工具，本项目不训练新模型。

一 八种表示的正式结果

数据来自 ChEBI-20 的 MolT5 再发布版，开发集和测试集由本项目划分。筛选后候选池含 431 个分子：无立体化学信息、分子量≥300 Da 且至少 2 环等。

Identity 比较共同的 200 个题对；其余四项各比较共同的 199 个分子。少 1 个是因为含铜分子不满足当前 Graph3D 构象优化流程；不换题，也不把转换失败当作模型答错。

输入表示	同分子判断 准确率 ↑ n=200	子结构识别 Macro-F1 ↑ n=199	元素计数 准确率 ↑ n=199	QED 估计 MAE ↓ n=199	氢键供体 准确率 ↑ n=199
SMILES	0.985	0.839	0.231	**0.111**	0.814
SELFIES	0.540	0.546	0.015	0.180	0.432
Our Code	**1.000**	0.515	0.191	0.139	0.427
img 二维结构图	0.990	**0.868**	**0.342**	0.119	**0.975**
Graph2D 二维文本	0.850	0.438	0.055	0.145	0.382
Graph3D 三维文本	0.980	0.571	0.070	0.170	0.387
CML	0.970	0.523	0.196	0.128	0.467
MolJSON	0.620	0.434	0.216	0.149	0.357

↑越大越好，↓越小越好；n 是题数。粗体只表示本列数值最好，不表示已通过统计检验证明优于其他表示。

二 五列分别在问什么

同分子判断（Identity）：两份输入是不是同一个分子？准确率 1.000 就是全部答对。元素计数：碳 C、氮 N、氧 O、硫 S 各有几个？四个数全部正确才算该题答对。

子结构识别：分子里有没有指定官能团，即醇、酯等局部原子组合？主分数对固定 10 类的 F1 等权平均，同时考虑误报和漏报；0.868 不能读成“86.8%的题全对”。

QED 估计：估计一个描述“类药性”的 0 到 1 分数。MAE 是与参考值的平均绝对差，例如参考值 0.60、回答 0.70，该题误差为 0.10。氢键供体（HBD）：能提供氢键中氢的位点有几个？数量完全正确才算对，不是数全部氢原子。

三 最值得汇报的发现

图片在子结构、元素计数和 HBD 三项数值最好；但元素计数准确率也只有 34.2%，不能说图片已经全面可靠。SMILES 的 QED 误差最小；Our Code 的 Identity 为 200 题全对，但其他四项表现均不及图片。

Graph3D 比 Graph2D 的子结构和元素计数更好，QED 误差却更大；三维坐标没有改善所有任务。

四 八种输入形式的区别

表中的八行是八种分子表达方式，不是八个 AI 模型。Our Code 是一种给模型阅读的构造代码；模型不会执行它来算答案。

表示	模型实际读到的内容
SMILES	用元素符号、括号和数字描述原子及连接关系的一行字符串。
SELFIES	用方括号词元及对应规则描述分子的另一种字符串。

表示	模型实际读到的内容
Our Code	用 Atom 和 Bond 等记录逐条列出原子与化学键的代码。
img	二维化学结构图片，模型直接看图。
Graph2D	包含原子、化学键和二维坐标的 MOL 连接表文本。
Graph3D	同类 MOL 文本，坐标来自计算生成的三维构象。
CML	用 XML 标签组织原子、化学键等信息的文本。
MolJSON	按作者规定的 JSON 格式组织分子信息的文本。

Graph2D 和 Graph3D 在本项目中都是文本，只有 img 是图片；Graph3D 不是让模型观看三维球棍图。

五 Our Code 错在哪里以及下一步怎么查

已经知道的现象：HBD 的 199 题中，Our Code 答对 85 题，图片答对 194 题。有 109 题是图片答对、Our Code 答错，反过来的情况为 0 题。元素计数中，图片对而代码错 55 题，反向 25 题。

这说明本批题目有明显的成绩差距，但还不知道原因。

值得检查的线索：原版 Our Code 多数原子的氢数需要模型根据化学键推断。图片中的 NH、OH 标签直接显示部分氢，但碳和不少氢也会省略。两种输入表达氢信息的方式不同。

因此，“把氢数写清楚后，模型会不会更容易判断 HBD”是一个待验证的问题。目前不能说缺少氢数就是失分原因，也不能说补上氢数一定最好。

下一步的验证办法：准备两个版本，只让其中一个把每个原子的氢数写清楚，其他内容保持不变，再让模型回答同一批开发集题目。这叫单因素对照；开发集用于试方案，未参与调试的新题用于验证效果。

当前进度：Our Code + H 候选已完成，431 个分子通过结构一致性检查。它只补充氢信息，不改变分子；尚未交给模型测试，所以没有新成绩，也没有放进前面的八行表。

六 与教授论文的关系和结论范围

我们参考《Rethinking Molecular Text Representations for LLMs》（arXiv:2606.03057v1）的比较思路，加入 Our Code、图片、MOL 二维和三维文本及 QED 任务。这是扩展实验，不是整篇论文的完整复现。

结论限于当前模型名、样本和提示条件；服务端未提供权重版本，Identity 暂作描述性比较。坐标文本成绩不能证明完整三维推理能力，QED/HBD 也不代表药物疗效或安全性。

本次补充：后续各页展示原论文方法、实际运行的核心代码与五项模型 prompt、逐题评分证据、参考答案重建，以及不用 API Key 的复算命令；最后一节给出 GitHub 发布 commit、从全新目录 clone 复算的结果与远端自动化检查的实际状态。

七 原论文如何进行比较

论文研究的是分子的写法怎样影响 LLM 表现。作者先固定任务与参考分子，把同一分子转换成不同文本表示，给模型相应输入和统一任务提示，再用独立的标准答案评价。模型的理解能力由回答成绩体现。

正文第 3 至 4 节介绍任务与实验；PDF 第 15 页的 Appendix C 和 D 给出数据和任务定义；第 16 至 17 页的 Appendix E 公开实际 prompt。本文依据这份 25 页 PDF，不声称已经取得或运行作者的完整源码。

比较项目	原论文	本项目
输入表示	9 种文本表示	8 种输入形式，包含 Our Code 和图片
任务与模型	8 项任务，16 个 LLM	5 项任务，一个模型 deepseek-v4.1-flash

比较项目	原论文	本项目
数据划分	ChEBI-20 官方 test；部分任务 ZINC250K	ChEBI-20 再发布 validation/test 合并后由本项目重划
结构门槛	MW ≥ 300 Da 且至少 2 环	沿用门槛，并要求 Our Code 可解析与一致往返
元素计数	每次询问一种元素	一次询问 C/N/O/S，四种全部正确才算对
官能团	逐次询问 5 类中的一类	一次问 13 类，固定 10 类参与主 Macro-F1
任务输出	逐步推理后给 boxed 答案	要求一个简短 JSON 对象
性质任务	LogP、TPSA、HBD、HBA	本轮 HBD；新增 QED 估计

因此，我们复用了“控制模型和题目、改变输入形式”的比较思路，并修改了任务、问法和实验范围。这是扩展实验。它不能直接继承原论文的全部结论。

原文正文对 LogP/TPSA 使用 Spearman 相关；Appendix D 另有 exact-match 表述，二者措辞不完全一致。本项目 QED 明确采用 MAE，不能写成论文原指标。

八 核心代码怎样把分子变成模型输入

整条流程是：原始分子 → 筛选并冻结题单 → 生成表示 → 组装 prompt → 模型回答 → 对照 RDKit 参考答案 → 算分。模型只接收表示和问题；参考答案留在程序一侧。

基础类在 research/01_implementation/molecule_core.py。Atom 保存一个原子的记录，Bond 保存两个原子之间的键，Molecule 把这些记录组成一个分子。以下是原代码的字段摘录，省略校验方法。

```
@dataclass(frozen=True)
class Atom:
    id: str
    element: str
    formal_charge: int = 0
    isotope: int = 0
    aromatic: bool = False
    explicit_hs: int | None = None
    atom_map: int | None = None


@dataclass(frozen=True)
class Bond:
    begin: str
    end: str
    bond_type: str = 'SINGLE'
```

id 是分子内部标签，element 是元素，formal_charge 是电荷，aromatic 表示是否芳香。explicit_hs=None 沿用隐式氢规则；整数表示确定的显式氢数。未在构造文本写出的字段采用默认值，不能把“省略字段”理解成“没有氢”。

真正的表示转换入口在 bmb/reps.py，最终执行基础类的 to_code。下面是该函数的实际源码，打印换行仅为排版。

```
def our_code_text(smiles: str) -> str:
    """The professor prototype's constructor text for this molecule."""
    try:
        return _core_molecule(smiles).to_code()
    except Exception as exc:
        raise RepresentationError('Our Code cannot express %r: %s' % (smiles, exc)) from
     exc
```

例如乙醇 CCO 产生三条 Atom 和两条 Bond。模型阅读这些构造代码，不执行 Python 来得到答案。这个教学分子太小，不满足正式复杂度门槛。

```
Molecule(
    atoms=(Atom(id='a0', element='C'),
           Atom(id='a1', element='C'),
           Atom(id='a2', element='O')),
    bonds=(Bond(begin='a0', end='a1'),
           Bond(begin='a1', end='a2')),
)
```

上面乙醇构造为了阅读合并了行；完整生成格式与全部方法见源码。教授原型的三份核心文件字节一致，SHA-256 以 55d314aa 开头；本次未改基础类。

九 模型实际收到的提示词

以下来自实际 bmb/tasks.py 及冻结历史输入，不是新编的优化提示。完整模板、八种表示说明和五个真实例子已导出到 prompts 目录。CodeBuddy 执行指令是开发工具的任务单，与这些分子题目不同。

所有任务共用的 **system prompt**

```
You are answering chemistry questions. You cannot run code, call tools, or look anything
    up. Answer only from the structure you are given. Reply with a single compact JSON
    object and nothing else.
```

意思是：只能根据给定结构回答，不运行代码或查询工具，只输出一个简短 JSON。模型的正确与否由外部程序判断。

Our Code 输入说明

```
Each molecule is given as Python source that builds explicit records. Atom(id=...,
    element=..., formal_charge=..., isotope=..., aromatic=..., explicit_hs=...,
    atom_map=...) describes one atom; Bond(begin=..., end=..., bond_type=...) describes
    one bond between two atom ids; Molecule(atoms=(...), bonds=(...)) collects them.
    Atom.id is only a local label inside that one expression and carries no chemical
    meaning. explicit_hs=None means the element's normal implicit-hydrogen rule applies;
     an integer is an exact hydrogen count.
```

user 消息的结构是 Representation 说明、Molecule 分子内容、任务 instruction；Identity 改为 A 和 B 两份内容。省略分子内容的模板不能冒称一条完整历史请求；完整请求在 historical_our_code_examples.json。

同分子判断的实际 **instruction**

```
Structure A and structure B are two encodings of possibly the same molecule.
Decide whether they describe the SAME molecule.
Two encodings describe the same molecule when they have the same atoms, the same bonds
    between those atoms, the same bond orders and the same formal charges, regardless of
     which atom the text happens to start from or how the records are ordered.
Reply with JSON only: {"same_molecule": true} or {"same_molecule": false}.
```

模型应返回 same_molecule 的 true 或 false；依据原子、键、键级和电荷判断，不比较字符串是否一样。历史输入排序规则仍有差异，所以该项不能作确认性优势结论。

官能团识别的实际 **instruction**

```
For the molecule below, decide for EACH functional group in the list whether it is
    present. Judge only from the structure given.
Groups: "aldehyde", "ketone", "ester", "carboxylic_acid", "amide", "primary_amine",
    "nitrile", "alcohol", "phenol", "benzene_ring", "alkyl_halide", "aryl_halide",
    "nitro"
Reply with JSON only, one boolean per group, for example {{"aldehyde": true}}.
```

实际问 13 类；主 Macro-F1 只对冻结的 10 类平均。aldehyde、nitrile、nitro 不进入主分数。原示例里的双大括号是已发现的格式缺陷，这里保留实际历史问法。

十 其余任务怎样问与怎样判分

元素计数的实际 instruction

Count how many atoms of each listed element are in the molecule below. Count heavy atoms only; do not count hydrogen atoms.
Elements: C, N, O, S
Reply with JSON only, one non-negative integer per element, for example {{"C": 0}}.
C、N、O、S 四个数全部与参考相同，该题才算正确。原示例同样含双大括号；未来修正需另立 prompt 版本，不能改写旧运行记录。

QED 估计的实际 instruction

Estimate the drug-likeness score QED (Quantitative Estimate of Drug-likeness) for the molecule below, on its usual scale from 0 (least drug-like) to 1 (most drug-like). Answer from the structure alone; do not run any code and do not attempt to compute descriptors externally.
Reply with JSON only: {"qed": <a number between 0 and 1>}.
模型估计 0 到 1 的 QED；程序计算回答与 RDKit 值的绝对差，再对题目平均得到 MAE，越小越好。该值不是疗效或安全性。

氢键供体计数的实际 instruction

Count the hydrogen-bond donors (HBD) in the molecule below, using the standard definition: the number of nitrogen or oxygen atoms that carry at least one hydrogen atom.
Reply with JSON only: {"hbd": <a non-negative integer>}.
HBD 问的是供体位点数量，不是分子内所有氢的数量。历史 prompt 用带氢的 N/O 作简化说明，oracle 则用 RDKit CalcNumHBD；特殊原子的定义边界应在新版本检查。

一个真实的历史失分例子

题目 t3-our_code-71464601 取自 main-02 原始日志；它是元素计数任务。完整 Our Code 输入、模型回答与另列参考答案见 prompts/historical_our_code_examples.json。

元素	模型回答	参考答案	是否一致
C	22	21	否
N	1	1	是
O	4	4	是
S	2	2	是

模型多算了一个碳。因此四元素 exact-match 规则下，该题得 0；不是给 3/4 分。这个例子证明如何产生失分，但不能单凭它确定为什么模型多算了碳。

十一 参考答案 请求与评分的核心代码

bmb/labels.py 产生参考答案。它使用独立 RDKit，不使用我们的基础类给自己的表示打分。以下是实际源码片段。

```
def atom_count(smiles: str, element: str) -> int:
    mol = Chem.MolFromSmiles(smiles)
    if mol is None:
        raise ValueError('RDKit cannot parse %r' % smiles)
    return sum(1 for atom in mol.GetAtoms() if atom.GetSymbol() == element)


def qed_reference(smiles: str) -> float:
    mol = Chem.MolFromSmiles(smiles)
    if mol is None:
        raise ValueError('RDKit cannot parse %r' % smiles)
    return float(QED.qed(mol))
```

官能团用 mol.HasSubstructMatch 匹配冻结 SMARTS 模式；HBD 用 rdMolDescriptors.CalcNumHBD。SMARTS 是指定局部原子和键组合的查询规则，定义在 config/functional_groups.frozen.json。

bmb/client.py 的_payload 把 prompt 放入真实请求，以下是实际请求体构造片段。extra_body 中的 enable_thinking 设置以及 response_format 也由该函数加入。

```
body = {
    'model': self.model_id,
    'messages': messages,
    'temperature': self.temperature,
    'top_p': self.top_p,
    'max_tokens': self.max_tokens,
    'stream': False,
}
```

历史记录为 deepseek-v4.1-flash、temperature=0、top_p=1、max_tokens=4096、zero-shot，请求 enable_thinking=false。run manifest 实际记录 Token Plan 通道；本次只离线复算，没有新调用模型。服务端未提供固定权重版本。

bmb/scoring.py 的 score_item 直接比较回答和参考。以下为函数中的真实分支；前面的 status 校验与其它分支在完整源码。

```
if task == 't3_atom_counts':
    return bool(parsed == label)
if task == 't4_qed':
    return abs(parsed['qed'] - label['qed'])
if task == 't5_hydrogen_bond_donors':
    return bool(parsed['hbd'] == label['hbd'])
```

子结构分数不是逐题 exact-match：程序分别累计每类的正确识别、误报和漏报，计算 F1，再对固定类别等权平均。共同题表由 build_common_table.py 先按每项任务取八种表示的交集，再重算指标。

十二 怎样复算并从 **GitHub** 审查

最短复算路径（路径 A）只需要 Python 3.10 或更高版本，不需要任何第三方包，也不需要 API Key。执行：

```
git clone https://github.com/CHANSingYeungSunny/Design-Molecule-Class.git
cd Design-Molecule-Class
python research/02_tests/test_baseline_core.py
python scripts/replay_table.py
```

四条命令按名称分工：test_baseline_core.py 验证教授提供的原子、键和分子基础类（输出 Ran 55 tests ... OK）；replay_table.py 重新解析保存的模型 JSON，用原评分器重算主表（输出 Historical replay passed）。输出在 reproduced/：summary.csv 是表格，per_item.jsonl 是逐题评分证据，verification.json 是校验结果。参考标签采用当时保存的 RDKit 值；这一步不是重新调用模型。

路径 B 才检验“参考答案本身能不能被重算”：装上 RDKit 等依赖后运行 scripts/rebuild_labels.py，它会用 RDKit 对 200 个 Identity 题对和 199×4 条理解题重新计算官能团、C/N/O/S 计数、QED 与 HBD，并与冻结值逐项比对（实测 996 条、0 条不一致）。只跑路径 A 或只看教学例子，不能说标签重建已经通过。

本次在本机 Windows 环境（Python 3.13.5、RDKit 2025.09.4）实测：核心测试 55/55；benchmark 自检 99 项 0 失败；客户端离线检查 14 项 0 失败；核心代码索引 53 行全部命中真实源码；40 份冻结题单、main 的 3000 项与 extension 的 4996 项输入 hash 通过；1200 张刺激图片字节校验通过；八行 40 格共同题结果与原 Word 一致，分母仍为 200/199/199/199/199；参考答案重算 996 条、0 条不一致。

原始行数与有效题数必须分开读：main-02 原始日志 3000 行且全部为 ok；extension-01 原始日志 5038 行，其中 38 条是 pending 历史占位、4 条属于已被当前冻结清单取代的历史题。复算按当前清单过滤并按题号取最后状态。

重新调用模型（路径 C）需要自备服务、key 和预算，并另用 run ID。即使 temperature=0，服务端权重版本或推理实现变化都可能让新回答不同，不能承诺重新推理逐题一致。

十三 GitHub 发布、固定链接与验收

仓库与实际 commit

https://github.com/CHANSingYeungSunny/Design-Molecule-Class

三个提交要分清：源码提交装的是代码、数据、冻结题单和原始响应，是复算与验收的对象；文档提交只改说明、脚本与报告；最终交付提交是推送后仓库的最新一条，它包含本文件。本文件不硬编码自己所在提交的 SHA，最终交付 SHA 另在交付清单中给出。

本报告所有代码、数据、模板的固定链接，都指向下面这个已经被 clone 并复算过的源码提交：aac92e78a7d1b1f80653d59ec45252a3a612e45a 。它不包含本报告文件本身。

已验证源码提交 aac92e78a7d1b1f80653d59ec45252a3a612e45a

说明文档与脚本修正提交：79a9f270673a3dd2a1945e6d19389cd06460e15e 。该提交只改动文档与脚本，不改动核心代码、冻结题单与原始响应。

文档修正提交 79a9f270673a3dd2a1945e6d19389cd06460e15e

提交前已逐项检查：未提交 .venv、local.env、任何密钥、第三方 RDKit 整仓与临时文件；暂存文件 1766 个中扫描不到形如密钥的字符串；教授基础类三份副本 SHA-256 仍为 55d314aa 开头。

每个入口能看到什么

README：三条复现路径、最短命令、已知限制

基础类 Atom / Bond / Molecule

实际模型 prompt 组装

导出的完整 prompt 模板

五项任务的真实历史请求与回答

独立 RDKit 参考答案

评分规则

无 API Key 历史复算入口

参考答案重建入口

main-02 原始模型回答

论文与项目逐条对应

核心代码索引与中文说明

审核结论与限制

从全新目录 clone 的实测结果

在与原工作目录分离的全新目录执行 git clone 后（远端实际 commit）：test_baseline_core.py 输出 Ran 55 tests ... OK；核心代码索引 53 行 OK；replay_table.py 输出 Historical replay passed: 40 frozen lists, all 40 table values match.，八行数值与第一节表格完全一致；rebuild_labels.py 重算 996 条参考答案、0 条不一致；reference/MolJSON 源码在 clone 后可正常导入并生成 MolJSON 文本。

必须讲清楚的界线：上述 clone 复算是本次工作在另一个目录执行的，没有另一位独立使用者的验收记录。仓库与 README 可供其他人自行复算，但“可供他人复算”不等于“已经由其他人独立验证”。

远端自动化检查的实际运行

工作流 Historical offline replay（GitHub Actions，ubuntu-24.04，Python 3.13，不含任何密钥）。以下 run 都用 gh api 读取过 head_sha、status 与 conclusion，不是凭“有成功 run”推断：

run ID	对应 head_sha	status / conclusion
38315294795	aac92e78…2e45a	completed / success
38316032354	10ef071c…23687	completed / success

Actions 运行 38315294795（对应源码提交 aac92e78）

Actions 运行 38316032354（对应文档提交 10ef071c）

最终交付提交对应的运行会在推送后单独核对，不能用上面两条代替本次验收。

排版与页数的说明

本报告由 python-docx 生成，DOCX 是流式格式。用本机 Microsoft Word 导出 PDF 时为 10 页；同一份文件在另一个渲染环境下页数可能不同。页数取决于渲染器的字体度量与页面设置，页数不是验收标准；验收看章节是否重复、40 格成绩是否未改、分母 200/199 与 13 问 10 评是否未变、链接是否可查。文中源码片段为排版做了换行处理，打印换行不等于可执行的完整源码，完整源码以仓库固定链接为准。

已完成与未完成的界线：已完成“从保存响应重算报告”“用 RDKit 重建参考答案并逐项比对”“发布到本人仓库并在分离的全新目录 clone 复算”。未执行“重新付费调用模型”，因为它需要自备服务与预算，且新回答不保证与历史逐题一致。

我向教授说明时应能解释：谁算标准答案（RDKit，在实验程序一侧）、模型收到什么（只有表示和问题）、是否执行代码（不执行，也没有工具通道）、一个回答如何算分（逐题判定后按任务汇总）、哪些改动来自论文、哪些是本项目选择，以及现有比较限制（Identity 描述性、13 问 10 评、199 分母的原因）。提交代码有助于审核，但理解这些过程仍需要我自己完成。